DIGITAL MINE / OPENGIZ 2026

OREFLOW KZ

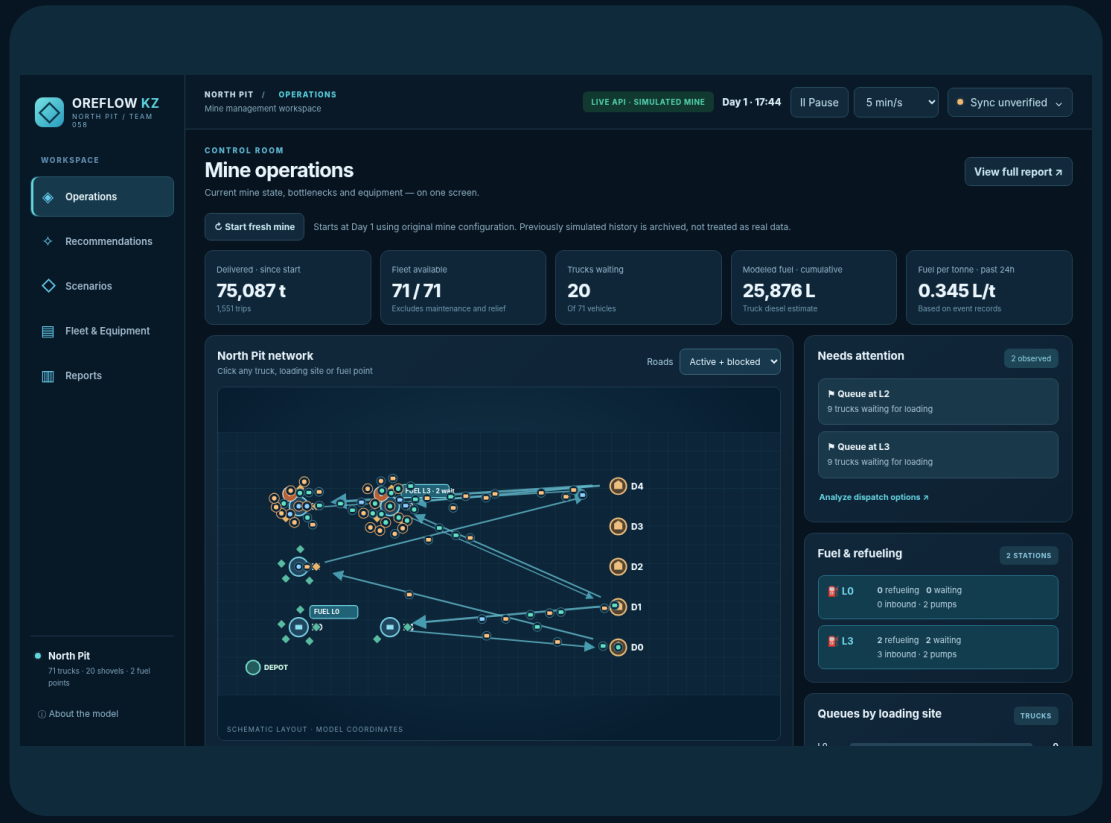
An open-pit twin built for decisions

Watch operational states. Detect bottlenecks. Test failures. Compare recovery plans.

71
TRUCKS

20
SHOVELS

159
DITTO TWINS



Zhenisbek Umaraliev | GitHub: jenisbekk

Team 058 / North Pit / Source-configured simulation on OpenEgiz

Model-generated operations; not field sensor measurements.

The operational decision

The problem is not truck animation; it is knowing which feasible response helps when a mine becomes congested or equipment fails.

Observe

Live queues, truck assignments, shovel states, fuel access and deliveries are tied to a common mine model.

Compare

Nearest, queue-aware and OreFlow policies use matched starting conditions; throughput and fuel trade-offs are visible.

Respond

An isolated scenario forecasts a breakdown or road closure; recovery alternatives can be tested without harming the live run.

Users

Dispatchers / Shift supervisors / Equipment planners

Decision: where to send available trucks, and what to change after a disruption.

A transparent North Pit configuration

Anonymized OpenMines input defines fleet classes and mine service capacity. Movement, diesel, queues and failures are synthetic calculations.

9 × 77 t trucks

Large-capacity class. 25 km/h nominal configured speed.

33 × 55 t trucks

Mid-capacity class; modeled travel, loading and fuel.

29 × 35 t trucks

Smaller-capacity class; heterogeneous loads create dispatch trade-offs.

5	5	20	2	55
loading sites	dump sites	shovels	fuel stations	directed roads

Input file: team/oreflow/data/north_pit_mine.json | OpenMines MIT | Schematic topology, not GPS

159 registered digital twins

The mine is represented by named Eclipse Ditto things. MQTT and history are part of the system—not decorative dashboard labels.

OreFlow engine

Event model + control

→

MQTT

Published twin updates

→

Eclipse Ditto

Persisted things/readback

→

Operator interface

Map, inspector & scenario views

Telegraf

↓ InfluxDB

↓ Grafana history

Twin inventory

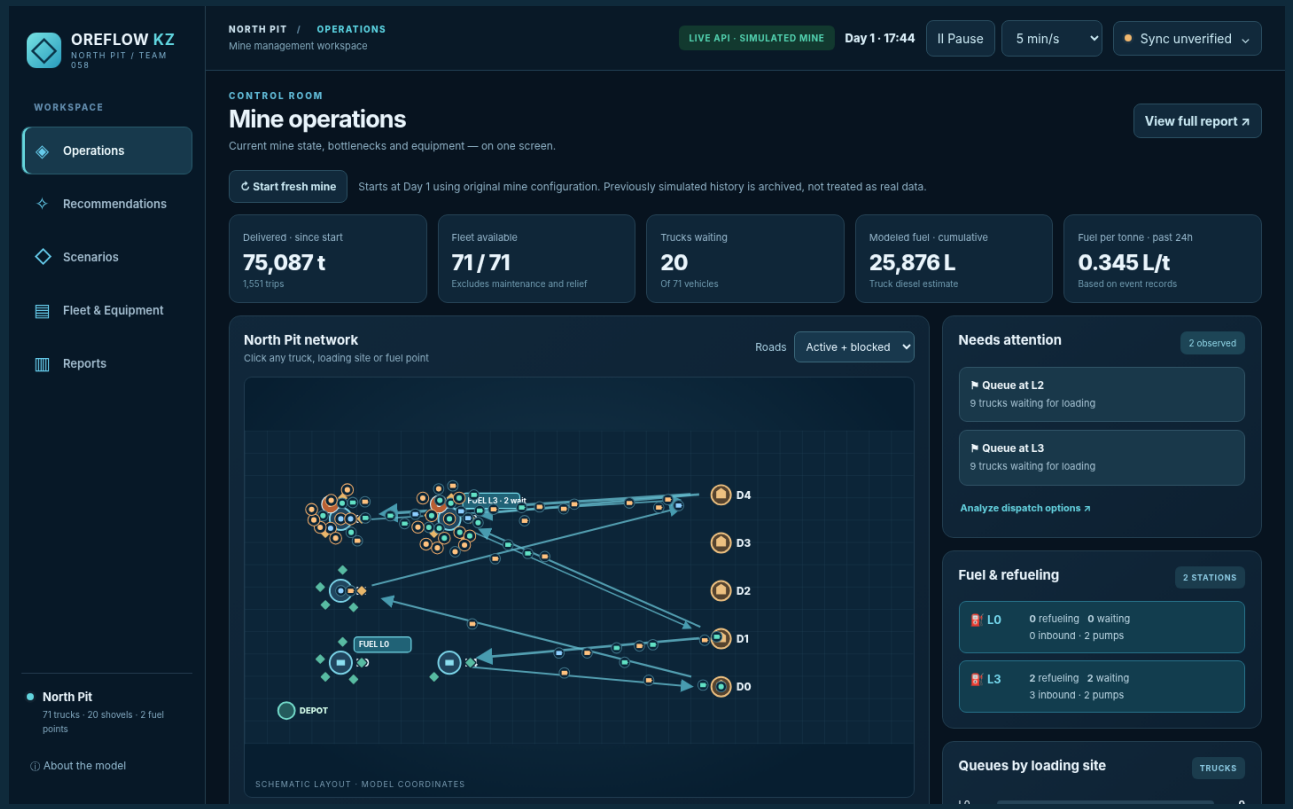
71 trucks, 20 shovels, 10 sites, 2 stations, 55 roads, 1 mine.

Evidence route

Inspect /api/ditto/t001 or /api/ditto/road-l0-d0; run tools/verify-oreflow.py on the actual Docker stack.

One operating mine, not five disconnected charts

The main view exposes equipment, active routes, queues, service points and modeled operational activity.



Spatial mine

Interactive schematic with sites, trucks and directed connections.

Inspect assets

Truck details open in-page; queue and fuel station status stays visible.

Screenshot: browser-rendered UI using recorded simulated API data.

From a breakdown to a testable response

Normal, disrupted and recovery outcomes are calculated from isolated simulation branches, not inserted into the active run.



1 / Choose

Shovel, road, fuel or fleet disruption; pick target and duration.

2 / Calculate

Compare delivery, fuel, waiting and affected assets.

Undo: simply leave or rerun.

Live mine remains unchanged.

Counterintuitive results are disclosed, not forced into a preset improvement claim.

Smart dispatch with visible trade-offs

Policies are reproducibly evaluated using common seeds and compatible equipment constraints—not a promise of real-world superiority.

Nearest

Simple distance-led baseline: useful comparator, vulnerable to capacity bottlenecks.

Queue-aware

Uses predicted congestion at loading resources rather than distance alone.

OreFlow policy

Tests resource completion time plus modeled fuel/queue trade-offs.

Credibility checks

Capacity

No truck can carry more than its configured class limit.

Infrastructure

Failed shovels and closed routes restrict new eligible work.

Fuel

Modeled refueling takes queue and pump capacity.

Evidence

Compare tonnes, waiting and litres/t — not just total diesel.

What is verified — and what is not

We separate source facts, simulated behavior, local tests and measurements from the live OpenEgiz installation.

Local software checks

Scenario isolation, CSV schemas, equipment definitions, frontend fixture interactions and smoke tests have been exercised.

Live platform checks

The owner observed current-run Ditto readbacks for mine/truck/fuel/road, 159 registered twins and historical InfluxDB telemetry.

Known limits and disclosure

No authenticated physical sensors, fleet GPS, road survey, mineral grades or calibrated diesel curves. Screenshot content and What-If forecasts are simulated. A completely clean judge VM startup has not been independently reproduced by the package author.

Reproduce the mine in the judge VM

Ubuntu 24.04 + Docker; no manually supplied credentials. The README is the single starting point.

```
$ bash scripts/oreflow-start.sh
$ bash scripts/oreflow-verify-ready.sh
```

01	**Operations**	Inspect spatial mine, truck and fuel station.
02	**Fleet & Equipment**	Read named asset state and Ditto readback.
03	**Recommendations**	Compare policies and bottleneck trades.
04	**Scenarios**	Compute shovel stop or road closure; review all outcomes.
05	**Reports**	Inspect labeled production and fuel CSVs.

Interface: http://localhost:8099 | Source code, limitations and exact procedures: README.md + SUBMISSION.md